Marwadi University	**Marwadi University** **Faculty of Technology** **Department of Information and Communication Technology**	
Subject: Digital VLSI Testing (01CT0629)	**Aim: To detect transition faults in a sequential circuit by applying test patterns for 0 → 1 and 1 → 0 transitions.**	
Experiment No: 15	**Date: 10-04-2026**	**Enrolment No: 92301733056**

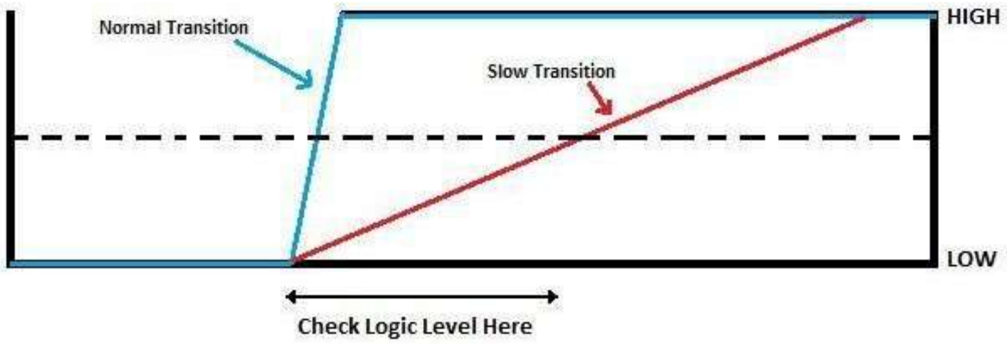
AIM: **To detect transition faults in a sequential circuit by applying test patterns for 0 → 1 and 1 → 0 transitions.**

IDE: EDA Playground

IDE: EDA Playground

THEORY:

Transition Faults occur when a signal fails to make a proper transition (0→1 or 1→0) within the required time, indicating a slow-to-rise or slow-to-fall fault. This test checks if transitions propagate correctly through combinational paths. We excite and propagate transitions using test vectors and observe if they appear at the outputs within expected delay.



Transition Fault

Verilog Code:

A. D Flipflop with delay:

```
// Code your design here
module D_ff_transition_fault(
  input D,
  input clk,
  output reg Q
);

  initial Q=0;

  //Sequential behaviour with delay
  always @(posedge clk)
    begin
      Q <= #2 D; //2ns delay

    end
     endmodule
```

B. Testbench

```
// Code your testbench here
// or browse Examples
`timescale 1ns/1ps

module tb_transition_fault;

reg clk;
reg D;
wire Q;

time start_time, end_time;
time delay;

// Instantiate DUT
D_ff_transition_fault uut (
    .clk(clk),
    .D(D),
    .Q(Q)
);

// Clock generation
always #5 clk = ~clk;

// Task: Check 0→1 transition (slow-to-rise)
task check_rise;
input max_delay;
begin
    start_time = $time;

    @(posedge clk);   // apply input at clock edge

    fork
        begin
            @(posedge Q);   // wait for rising transition
        end
        begin
            #10; // timeout protection
        end
    join_any
    disable fork;

    end_time = $time;
    delay = end_time - start_time;

    if (Q !== 1)
        $display(" LOGIC ERROR (0→1): Q=%b", Q);
    else if (delay > max_delay)
        $display(" SLOW-TO-RISE FAULT: Delay=%0t > %0t", delay, max_delay);
    else
        $display(" PASS (0→1): Delay=%0t", delay);
end
```

```
endtask

// Task: Check 1→0 transition (slow-to-fall)
task check_fall;
input max_delay;
begin
   start_time = $time;

   @(posedge clk);

   fork
      begin
         @(negedge Q);   // wait for falling transition
      end
      begin
         #10; // timeout protection
      end
   join_any
   disable fork;

   end_time = $time;
   delay = end_time - start_time;

   if (Q !== 0)
      $display(" LOGIC ERROR (1→0): Q=%b", Q);
   else if (delay > max_delay)
      $display(" SLOW-TO-FALL FAULT: Delay=%0t > %0t", delay, max_delay);
   else
      $display(" PASS (1→0): Delay=%0t", delay);
end
endtask

// Test sequence
initial begin
   // Waveform dump
   $dumpfile("dump.vcd");
   $dumpvars(0, tb_transition_fault);

   clk = 0;
   D = 0;

   #10;
   $display("\n--- Transition Fault Detection Start ---");

   // Test 0 → 1
   D = 1;
   check_rise(3);

   // Test 1 → 0
   #10;
   D = 0;
   check_fall(3);

   // Repeat tests
```

```
    #10;
    D = 1;
    check_rise(3);

    #10;
    D = 0;
    check_fall(3);
    $display("\n--- Testing Completed ---");
    $finish;
  end
  endmodule
```

SIMULATION RESULTS:

CONCLUSION:

The experiment successfully demonstrated the detection of transition faults in a sequential circuit by applying appropriate two-pattern test vectors for both 0→1 and 1→0 transitions. By operating the circuit under clocked conditions, it was observed that faults such as slow-to-rise and slow-to-fall can be identified only when signals fail to switch within the required time. This highlights the importance of at-speed testing in sequential circuits, as timing-related defects directly affect circuit performance. Thus, the procedure effectively validates the presence of transition faults and emphasizes the role of proper test pattern generation in ensuring reliable circuit operation.

Post Lab Exercise:

1. How many test patterns (vectors) are required at minimum to detect a single transition fault on a specific node?

   To detect a single transition fault (slow-to-rise or slow-to-fall) at a specific node, a minimum of **2 test patterns (vectors)** are required because transition faults are time-dependent. The first vector initializes the node to a known logic value (0 or 1), and the second vector is applied to create a transition (0→1 or 1→0) at that node and propagate the effect to the output for observation. Hence, transition fault testing always requires a **two-pattern test**.

2. In the context of transition faults, what is the specific name of the fault where a node fails to change from 0 to 1 in time?

   In the context of transition faults, when a node fails to change from **0 to 1 within the required time**, the fault is specifically called a **slow-to-rise (STR) fault**. This type of delay fault indicates that the rising transition is too slow, potentially causing incorrect operation in high-speed circuits.

3. Which clock edge in the testbench above is responsible for "launching" the transition from the input to the internal logic?

   In a typical transition fault testbench, the **first active clock edge (usually the rising edge)** is responsible for **launching the transition**. This edge captures the initial vector and triggers the change at the input, starting the transition that propagates through the internal logic.

4. Why is a sequential circuit more susceptible to transition faults than purely combinational logic?

   A **sequential circuit** is more susceptible to transition faults because it depends on **clock timing and storage elements (flip-flops)**. Any delay in signal transitions can violate setup and hold time constraints, leading to incorrect data being latched. In contrast, purely combinational circuits do not rely on clocked storage, so while delays may affect speed, they are generally less likely to cause functional errors unless timing constraints are critical.

5. What is the difference between a "Stuck-at Fault" and a "Transition Fault" regarding the speed of the clock during testing?

   The main difference is in the **clock speed used during testing**: a **stuck-at fault** can be detected using a **slow (low-frequency) clock** because it only checks whether a node is permanently stuck at 0 or 1, independent of timing. In contrast, a **transition fault** requires a **high-speed (at-speed) clock** because it tests whether a signal can transition (0→1 or 1→0) within the required time; therefore, it must be tested under normal operating speed conditions to detect delay-related defects.